



Navigating ARC-AGI: From Zero to One

An interactive guide to the theory, implementation, and state-of-the-art strategies for the ARC-AGI Challenge.

"Easy for Humans, Hard for AI" — the defining characteristic that makes ARC-AGI a powerful benchmark for general intelligence.

fluid intelligence

program synthesis

test-time training

induction · transduction

refinement loops

ARC-AGI-1 / 2 / 3



A Note from a Fellow Beginner

Hello! Like many others, I was fascinated by the ARC-AGI challenge but found the learning curve a bit steep. I created this guide as a personal project to connect the dots for myself.

My goal is simple: to provide a single, clear starting point for newcomers by synthesizing the core concepts into one easy-to-follow narrative. If you're just starting your ARC journey, I hope this resource helps. Happy Journey!



What changed (updated mid-2026)

I first wrote this during the 2024 Kaggle season. A lot has happened since, so I went back through the whole thing. The short version:

- ARC-AGI-1 has basically fallen — frontier models now sit around 90%+ on it, so it's more of a warm-up than the real test.



- ARC-AGI-2 became the actual fight. A full competition (ARC Prize 2025) ran on it, and the gap between "a model with unlimited compute" and "a solution that fits inside Kaggle's limits" turned out to be the whole story.
- There's now an **ARC-AGI-3**, and it's a completely different beast: agents dropped into little games with no instructions. Every frontier model scores under 1%.
- The research mood shifted from "scale it up" to "refine it" — small recursive models and feedback loops did surprisingly well.

I've updated the numbers, added the 2025 winners, and written up the newer ideas. If something here looks out of date by the time you read it, well — that's ARC for you. It moves fast.

What is ARC-AGI?

ARC-AGI is a benchmark for fluid intelligence — how well a system handles a problem it has never seen before. Most benchmarks reward what a model already knows; this one rewards how fast it can pick up something new.



Novel Puzzles

Each task is unique and designed to resist memorization.

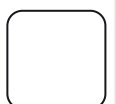


Skill Acquisition

Tests the efficiency of learning new skills, not just performance.



Core Knowledge



Uses only universal cognitive primitives for fair comparison.

Part I: Foundations

The Philosophy and Design of ARC-AGI

A REAL ARC TASK

007BBFB7 ↗

→

→

→

TEST — PREDICT THE OUTPUT

→

?

Straight from the ARC-AGI training set. Same hidden rule in all three pairs — can you predict the last grid?

The Abstraction and Reasoning Corpus for Artificial General Intelligence (ARC-AGI) comes with a strong opinion about what intelligence actually is. François Chollet built it because he thought the field was measuring the wrong thing.

Intelligence is how fast you learn, not what you already know

The core idea is that intelligence isn't demonstrated by the **possession** of a specific skill, but by the **efficiency of acquiring new skills** when faced with a novel problem. That's almost the opposite of what most AI benchmarks reward, which is performance on tasks you can master by training on enough data.

When that's the game, a high score can just be bought with more data and compute, and it tells you little about whether the system can actually adapt. An AI that plays superhuman Go has mastered Go; it hasn't necessarily gotten any smarter in general.

Chollet's definition is precise: *"The intelligence of a system is a measure of its skill-acquisition efficiency over a scope of tasks, with respect to priors, experience, and generalization difficulty."* ARC-AGI is that definition turned into a test — every task is one-of-a-kind, so memorizing or pattern-matching against a training set won't get you there.

To make that measurable, every puzzle follows the same shape — the [ARC Task Format](#) — handing you a few examples to work the rule out from.

Core Knowledge Priors: keeping the comparison fair

For the human-vs-AI comparison to mean anything, the test has to lean on **fluid intelligence** — the ability to reason, solve novel problems, and adapt to new situations — and not **crystallized intelligence**, which relies on accumulated knowledge and skills. If a task needed you to know history or English, it would just favor whoever was pre-trained on the right material, and you'd be measuring prior exposure instead of reasoning.

ARC-AGI gets around this by relying on only a small set of **Core Knowledge Priors** — cognitive building blocks that are either present at birth or picked up very early in life, with minimal explicit instruction.



Key Concept: Core Knowledge Priors

● Objectness

The ability to perceive a scene in terms of discrete objects with properties like cohesion (objects move as wholes) and persistence (objects don't randomly appear or disappear).

▢ Basic Topology & Geometry

Intuitive understanding of connectivity, symmetry, inside/outside relationships, and distance.

1 2 3 4 Elementary Number Sense

Simple counting and basic integer arithmetic.

🎯 Goal-Directedness

The notion that actions are taken to achieve goals.

Keeping the required knowledge down to these few universal primitives means a good score has to come from **actually reasoning**, not from what you happened to know going in. The public training set is put together to walk you through every prior you'll need for the evaluation tasks — think of it as the tutorial level.

One Benchmark Becomes Three: ARC-AGI-1 → 2 → 3

Here's something that trips up newcomers: "ARC-AGI" isn't one test anymore. There are now three, and each one was born the moment AI got too good at the last. That pattern—build a test, watch it get solved, build a harder one—is the whole rhythm of this field.

ARC-AGI-1 (2019) was the original. It did its job for years, but it had a weak spot: a big chunk of it could be cracked by brute-force program search, which says more about compute than about reasoning. By the end of 2024, OpenAI's o3 cleared the human bar on it, and by 2026 most frontier models score in the 90s. It's effectively solved now—still useful as a warm-up, but no longer the real challenge.

ARC-AGI-2 (March 2025) is the current battleground and what the prize money is chasing. It keeps the same format and philosophy but is built specifically to defeat brute force and to target the things modern reasoning models are bad at. It was also calibrated

against real people—a panel of 400+ testers in San Diego, with every kept task solvable by at least two humans in two tries. The average person scores around 60–66%; the panel together gets 100%.

ARC-AGI-3 (preview 2025, full launch March 2026) threw out the format entirely. Instead of static grids, you get dropped into a tiny turn-based game with no rules, no goal, and no instructions, and you have to figure it all out by playing. It's covered in its own section below. The headline: humans breeze through it, and every frontier AI scores under 1%.

New Conceptual Hurdles in ARC-AGI-2

Based on the failures of frontier AI models, ARC-AGI-2 introduces tasks that test for new, more complex reasoning abilities:

- Symbolic Interpretation:** Tasks where visual symbols must be interpreted as having semantic meaning beyond their shape, such as a shape representing an action.
- Compositional Reasoning:** Tasks that require discovering and applying multiple, interacting rules simultaneously.
- Contextual Rule Application:** Tasks where the correct rule to apply depends on the specific context within the grid, moving beyond superficial global patterns.

The ARC-AGI Family at a Glance (2026)

Feature	ARC-AGI-1	ARC-AGI-2	ARC-AGI-3
Launched	2019	March 2025	March 2026
Format	Static grid in → out	Static grid in → out (harder)	Interactive turn-based games
Tests for	Generalization, basic abstraction	Symbolic, compositional & contextual reasoning + <i>efficiency</i>	Exploration, goal-finding, on-the-fly learning

Feature	ARC-AGI-1	ARC-AGI-2	ARC-AGI-3
Brute-force-able?	Yes (its weak spot)	No (by design)	N/A — different paradigm
Human baseline	High	~60–66% individual / 100% panel	Humans solve ~all of it
Best AI (2026)	~90–96% (solved)	~85% unconstrained / ~24% under contest limits	<1%
Status	Retired to "warm-up"	The main event	Wide-open frontier

Note the two very different numbers for ARC-AGI-2. That gap—between what a model can do with unlimited compute versus what fits inside the competition's offline budget—is the single most important thing to understand about where the field is right now. More on that just below.

The ARC-AGI Ecosystem: Datasets and Evaluation

To actually compete, you need to know the practical side: which datasets exist, how scoring works, and where the community lives.

Navigating the ARC Datasets

The data comes in a few separate sets, each with its own job. Using them the right way matters — both for building your solver and for trusting the numbers you get back.

Overview of ARC-AGI-2 Datasets



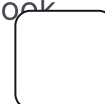
Dataset Name	Number of Tasks	Purpose	Access	Key Considerations
Public Training	~1,000	Training algorithms, learning Core Knowledge priors.	Public	Contains easier, "curriculum-style" tasks. Use freely for development.
Public Evaluation	120	Final local evaluation of an algorithm.	Public	Do not use for iterative development. Treat as a one-shot evaluation.
Semi-Private Evaluation	120	Powers the public leaderboard on arcprize.org.	Private (Kaggle)	Used to test both open and closed-source models.
Private Evaluation	120	Official ranking for the Kaggle prize competition.	Private (Kaggle)	The ultimate test of generalization. No internet access allowed.

Understanding the Rules of the Game

Competition Rules

Scoring (pass@2): For each test input you get two attempts. If either is a pixel-perfect match, the task counts as solved. (ARC-AGI-3 is scored differently — by how efficiently your agent acts compared to a human.)

The Kaggle box (this is the important one): prize submissions run offline in a Kaggle notebook — **no internet**, fixed hardware, hard time limit. In 2025 that was roughly 12 hours on 4× L4 GPUs. No internet means you can't call GPT, Claude, or Gemini in the prize track; your whole solution has to ship inside the notebook.



Open source: to win money you must release a complete, reproducible solution. The 2026 cycle asks for a true public-domain license (CC0 or MIT-0). This is the rule that turns each year's winner into next year's starting point.

The current live competition is **ARC Prize 2026**, with **\$2,000,000** in prizes split across three tracks: a **\$1M** track for cracking ARC-AGI-2 (the 85% Grand Prize, still unclaimed), a milestone-based track for building ARC-AGI-3 agents, and a **\$450K** paper track for new ideas. All three want open-source work.



The "two scores" trap (read this before quoting any number)

Every time you see an ARC-AGI score, ask one question: *was it measured with unlimited compute, or under the contest budget?* They are wildly different.

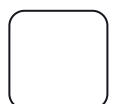
Public leaderboard (unlimited)

A frontier model allowed to spend freely. On ARC-AGI-2 in 2026, the top systems reach the low-to-mid 80s — but at \$10–30+ per task.

Kaggle competition (budgeted)

Offline, fixed hardware, a few cents per task. The 2025 winner landed at **24%** for about \$0.20/task. This is what the prize is actually judged on.

So when a headline says "AI hits 85% on ARC-AGI-2," it's usually the left column. The Grand Prize—85% *under the budget*—is still wide open. A fair warning: a lot of third-party "leaderboard" sites blur these two together, so when in doubt, trust arcprize.org's own verified leaderboard.



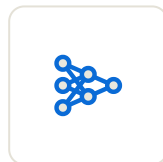
Part II: Core Methodologies

The Evolution of ARC-AGI Approaches



Program Synthesis

Infer explicit rules from examples



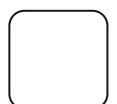
Neural Networks

Guide search with learned intuition



Test-Time Adaptation

Adapt dynamically to each task





Refinement Loops

Guess, check, fix — repeat

Each step didn't replace the last — the best 2026 systems stack all four.

Program Synthesis and Domain-Specific Languages (DSLs)

Program synthesis is the oldest serious approach to ARC, and it goes straight at the heart of the problem: work out the rule from the examples. The idea (also called inductive programming) is to automatically write a program that satisfies a spec — and here the spec is just the demonstration pairs.

So you're hunting for a program P that turns every training input into its matching output. Find one, assume it captures the rule, and run it on the test input to get your answer.

This is the **inductive** route: pin down a general rule first, then execute it. The other route, **transduction**, skips the program and predicts the output directly from the examples. Program synthesis ran the show in ARC's early years — the 2020 Kaggle winner was built this way.

The Power of DSLs

The hard part is that the space of possible programs is enormous, which is where a **Domain-Specific Language (DSL)** comes in. A DSL is a small language built for ARC: a hand-picked set of functions, or **primitives**, for common grid operations. The whole game is balance — expressive enough to actually solve tasks, small enough that searching it stays feasible.

Common DSL Primitives:

rotate_grid

find_objects

mirror_object

count_colors



draw_line

shift_object

DSL Program Example

solve_5521c0d9.py

```
# Simplified representation of the solver for task 5521c0d9 from Hodel's arc-dsl
def solve_5521c0d9(I):
    # 1. Extract all non-background objects from the input grid 'I'.
    objs = dsl.objects(I, univalued=True, diagonal=False, without_bg=True)

    # 2. Merge all extracted objects into a single 'foreground' object.
    foreground = dsl.merge(objs)

    # 3. Create a new grid by removing the foreground, leaving only the background.
    empty_grid = dsl.cover(I, foreground)

    # 4. Create a function 'offset_getter' that calculates an upward shift vector
    #     equal to an object's height. This is done by composing three functions:
    #     height → invert → toivec (get height, negate it, convert to vector).
    offset_getter = dsl.chain(dsl.toivec, dsl.invert, dsl.height)

    # 5. Create a function 'shifter' that takes an object and moves it.
    #     The 'fork' primitive applies the 'shift' operation, using the object
    #     itself as the first argument and the result of 'offset_getter(object)'
    #     as the second argument.
    shifter = dsl.fork(dsl.shift, dsl.identity, offset_getter)

    # 6. Apply the 'shifter' function to every object in the 'objs' list
    #     and merge the results into a single object of shifted shapes.
    shifted = dsl.mapapply(shifter, objs)

    # 7. Paint the final 'shifted' object onto the 'empty_grid'.
    0 = dsl.paint(empty_grid, shifted)

    return 0
```

Combined primitives in action

You can read this one top to bottom — no black box, just a short program: pull the grid apart into objects (`dsl.objects`), work out how far to shift each one, move them (`dsl.mapapply`), and paint the result back (`dsl.paint`). The catch is that a synthesis

system has to *find* this exact seven-step sequence among an enormous number of possibilities.

The Power of Search: From Brute Force to Neurally-Guided

Once you have a DSL, solving a task turns into a search problem: find the sequence of primitives that does the job.

The Combinatorial Explosion

Even with a constrained DSL of 100 primitives, the number of possible programs grows exponentially:

100
Length 1

10K
Length 2

1M
Length 3

100M
Length 4

Beyond Brute Force: Smarter Search

Modern solvers use smarter search to get through that space without checking everything:

Classic & Modern Search

Monte Carlo Tree Search (MCTS): Balances exploration and exploitation to find promising program paths.

Adaptive Branching MCTS (AB-MCTS): An advanced variant from Sakana AI that adaptively decides whether to search deeper (refine) or wider (explore).

Beam Search & Heuristic Search: Methods that use rules of thumb or maintain multiple candidate programs to guide search toward likely solutions.



Neural Guidance

GridCoder: Uses a Transformer to predict the most likely sequence of DSL primitives, guiding the search probabilistically.

Execution-Guided Search: A neural network learns a distance metric between grids to evaluate which intermediate step is "closest" to the goal.

Learning Program Space (LPS): The main GridCoder approach where a model predicts the final program directly.

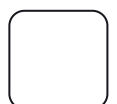
The big step forward was letting a neural network steer the search — turning it from "try everything in order" into something closer to "learned intuition" about which paths are worth following. It's faster, and a lot closer to how people actually search.

The 2025 twist: evolve the program, and let the searcher teach itself

Two ideas pushed search forward in the last competition. The first is **evolutionary search**: instead of building a program once, you keep a small population of candidate solutions and have an LLM mutate and recombine the best ones, generation after generation. Jeremy Berman rode this to the top of the boards — and made one counterintuitive discovery along the way. He stopped evolving *Python* and started evolving plain-English descriptions of the rule. Turns out natural language is easier for an LLM to tweak and refine than code, and it got him to about 29% on ARC-AGI-2 for roughly \$8 a task.

The second is **self-improvement**. The award-winning *SOAR* method runs evolutionary search, then does something clever with the wreckage: every failed program is secretly a *correct* program for some *other* task (whatever it actually computed). Relabel those failures as solved examples, fine-tune the model on them, and search again — a little smarter each round. It climbed to ~52% on ARC-AGI-1 with no hand-written DSL at all. The theme to notice: in both cases, the magic isn't a single brilliant guess, it's the loop.

Test-Time Adaptation: The Modern Paradigm



Program synthesis ruled the early years, but the breakthroughs of 2024 came from **Test-Time Adaptation (TTA)** — letting the model adjust to each task, at inference time, using that task's own demonstration pairs. By 2024 every top solution had some form of it.

TTA splits into two flavors. Test-Time Scaling (TTS) spends more compute at inference but leaves the weights alone; Test-Time Training (TTT) briefly fine-tunes the model on the spot.

Test-Time Scaling (TTS)

TTS just throws more compute at inference, *without* touching the weights. That can be as simple as sampling lots of answers and keeping the best, or as involved as chain-of-thought reasoning or Sakana AI's **Adaptive Branching Monte Carlo Tree Search (AB-MCTS)**.

Test-Time Training (TTT)

TTT actually nudges the weights at inference time — a few steps of gradient descent on the handful of demo pairs for the task in front of it, then it throws the update away. MIT's team pioneered it for ARC, and it became the backbone of several top 2024 solutions.

The Test-Time Training (TTT) Workflow

1

Get Task

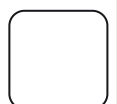
Receive a novel ARC task with a few train/test examples.

2

Augment Data

Expand the small demo set using symmetries (rotations, flips, color swaps) to create a temporary training set.

3



Train LoRA

Rapidly fine-tune a small LoRA adapter on the augmented data, leaving the base model frozen for efficiency.

4

Infer & Ensemble

Predict the output. Often done under multiple augmentations and combined via majority vote for robustness.

The Duality of Induction and Transduction

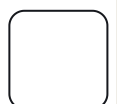
There's a split in how you can solve an ARC task, made precise in the prize-winning paper by Li et al. It lines up with [System 1 and System 2 thinking](#) — the fast-and-intuitive versus slow-and-deliberate idea from cognitive science. It's worth getting, because the strongest solvers use both at once.

Induction (Program Synthesis)

The **System 2**, deliberate path. First you work out an explicit program `f` that explains the training examples, then you run it on the test input: `y_test = f(x_test)`. The DSL search from earlier is inductive.

Excels at: Tasks requiring precision, multi-step logic, compositionality, and explicit computation.

Transduction (Direct Prediction)



The **System 1**, intuitive path. You predict the test output `y_test` directly, taking in the training pairs `(x_train, y_train)` and the test input all at once, without ever writing down a program. The LLM-based TTT methods are mostly transductive.

Excels at: Tasks relying on "fuzzy" perception, pattern completion, and holistic transformations.

*The very best ARC solutions are **ensembles** — they run both the inductive and the transductive solver, since the two tend to fail on different tasks.*

Refinement Loops: the idea that tied 2025 together

If you only remember one thing from the last competition, make it this. When the ARC Prize team looked back at everything that worked in 2025, almost every strong system — tiny models, giant models, program searchers — was doing the same thing underneath. They called it the [refinement loop](#), and summed it up with a line worth sitting with: *refinement is intelligence*.

The shape is simple, almost embarrassingly so:



Propose

Generate a candidate (a program, a grid, a guess).



Check

Score it against a signal you trust — for ARC, the demo pairs.





Fix

Keep what worked, repair what didn't, go again.

What makes ARC perfect for this is that it hands you a free, trustworthy checker: a candidate either reproduces the demonstration pairs exactly or it doesn't. That verifiable signal is *why* looping works so well here — and, honestly, why it's hard to copy this trick into messier domains where "correct" is fuzzy.

The loop shows up at three levels, and the best teams used all of them at once:

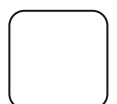
While making training data — generate synthetic puzzles, throw out the ones that don't pass a spec, keep the rest. The 2025 winner built hundreds of thousands of tasks this way.

While answering — a model that revises its own output step by step (the tiny recursive models below live entirely here), or a reasoning model talking itself out of a wrong answer.

While picking a final answer — generate many candidates under different rotations/recolorings and vote.

The practical takeaway

Don't ask your model for an answer. Ask it for a *process*. A modest model wrapped in a good guess-check-fix loop will usually beat a stronger model used in a single shot. There are even off-the-shelf "refinement harnesses" (Poetiq is one) that wrap a frontier API this way — they roughly took Gemini 3 Pro from 31% to 54% on ARC-AGI-2, just by looping.



The Plot Twist of 2025: Tiny Models That Punch Way Above Their Weight

Here's the result nobody saw coming. While everyone assumed you'd need a bigger and bigger model, three of the most talked-about papers of 2025 went the opposite way — and the paper prizes went to *all three* of them. On ARC, it turns out depth of *thinking* beats sheer size.

TRM — Tiny Recursive Model

~7M params · 1st paper, \$50K

A 2-layer network — about **7 million** parameters, less than 0.01% the size of a frontier LLM — that just keeps refining its own answer, around 16 passes deep. It hits ~45% on ARC-AGI-1 and ~8% on ARC-AGI-2, beating models ten-thousand times larger, and runs on a single gaming GPU. It's the cleanest proof that the refinement loop, not the parameter count, is doing the work.

HRM — Hierarchical Reasoning Model

~27M params · cautionary tale

A 27M-param model with two "brain-inspired" recurrent modules that made a big splash. Worth knowing for a second reason, though: when the ARC Prize team dug in, they found most of its ARC performance came from the *outer refinement loop and data augmentation*, not the fancy hierarchy. A plain transformer of the same size matched it. Lesson for your own work — ablate before you credit the clever part.

CompressARC — ARC without pretraining

~76K params · 3rd paper

The most extreme of the bunch: **76,000** parameters, no pretraining, no dataset, no search. For each puzzle it just runs gradient descent to find the shortest description that explains the demos (the Minimum Description Length idea — good compression *is* understanding). ~20 minutes a puzzle on one GPU, ~20–34% on ARC-AGI-1. It's a striking argument that you may not need a giant pretrained model at all.

One more thread worth flagging, because it'll matter going forward: a 2025–26 line of work argues we've been framing ARC wrong by feeding grids to language models as text. "[ARC Is a Vision Problem!](#)" treats each grid as an image and trains a vanilla vision transformer on it — these are puzzles you *look* at, after all. Keep an eye on the vision-first crowd.

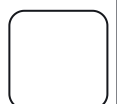
The Interactive Frontier: ARC-AGI-3

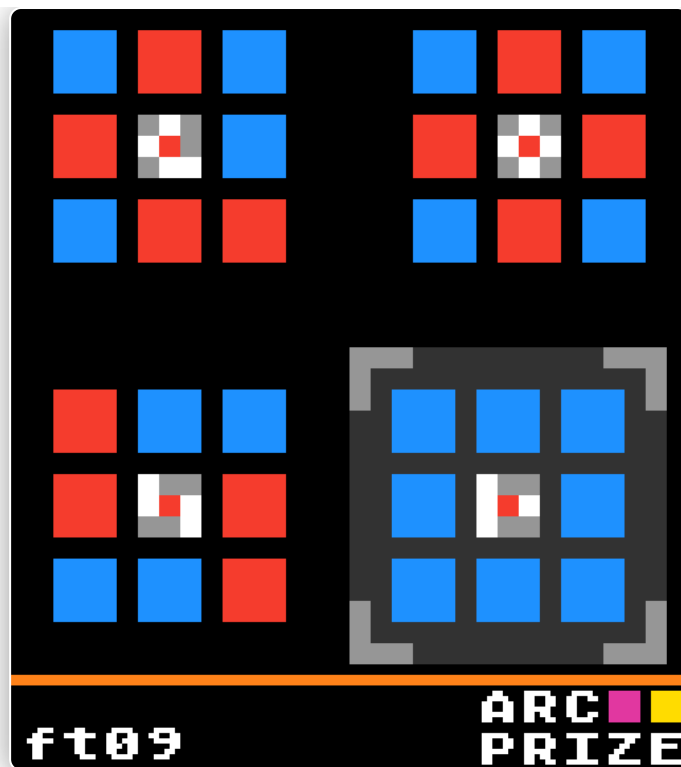
Everything so far has been about static puzzles: here are a few examples, predict the answer. In March 2026 the ARC Prize Foundation changed the game entirely — and if you want to work on something genuinely unsolved, this is where to look.

ARC-AGI-3 drops an agent into a small, turn-based game. **No instructions. No stated goal. No rules.** Just like being handed a video game you've never seen, you have to poke at it, watch what happens, work out what you're even trying to do, and then do it. It's the first *interactive* reasoning benchmark.

ARC-AGI-3 · REAL FOOTAGE FROM THE GAME "FT09"

play a real game ↗





No instructions, no stated goal. You just act, watch what changes, and work out for yourself that the boxed grid has to be made to match the pattern. Obvious once you see it – yet every frontier model still scores under 1% on games like this.

▸ WHAT THE AGENT WAS ACTUALLY THINKING

“If we look at the first three completed grids, each contains a small 3×3 pixel pattern in its center... the positions of the black pixels directly dictate which of the outer 6×6 blocks get colored light blue instead of dark blue. The yellow border acts as a ‘Submit’ button for the quadrant.”

– A REAL FRONTIER MODEL PLAYING FT09, QUOTED VERBATIM FROM THIS REPLAY. IT REASONED OUT THE RULE, TOOK 102 ACTIONS, AND STILL CLEARED JUST 1 OF THE GAME’S 6 LEVELS.

What it's actually testing

Static ARC tests whether you can spot a pattern. Interactive ARC tests the messier, more human skills that come before that:

Exploration

Can you poke at the world efficiently to learn how it works, instead of flailing randomly?

Goal-finding

Can you figure out what you're supposed to be doing when nobody tells you?

Memory

Do you remember what you tried, and reuse it, instead of repeating yourself?

Learning on the fly

Do you get *better* the longer you play a single game?

And the scoring reflects that shift: you're not graded on a final answer but on **how efficiently you act compared to a human**, averaged over a set of roughly 135 hand-built games. A perfect 100% means you match human efficiency everywhere.

The scoreboard right now is brutal

At the March 2026 launch, **every** frontier model scored under 1% — GPT-5.x, Gemini 3, Claude Opus 4.x, Grok 4, all of them, basically zero. Meanwhile humans (1,200+ testers across 3,900+ playthroughs) solved nearly all of it, and a lot of them said it was *fun*.

That's the original ARC promise — easy for people, hard for machines — back at full strength. The same models that ace ARC-AGI-1 fall apart the moment they have to explore a world instead of being handed the question.

What the early agents did

The preview round was small, and the best agent still only reached about **12.6%** efficiency — but the approaches that worked tell you where to start:

Predict what your actions do. The winning agent (StochasticGoose) trained a small CNN with reinforcement learning to guess which buttons actually change the screen, so it could explore on purpose instead of at random.

Build a map. The runner-up kept a graph of game states and pruned moves that led nowhere or looped back.



Plain LLM agents struggled. Dropping a big language model in and asking it to play mostly burned through actions. This is an RL / world-model problem, not a prompt-engineering one.

Why this might be your best bet

ARC-AGI-2 is a crowded, well-mapped engineering race. ARC-AGI-3 is the opposite — almost nothing works yet, the best score is ~12%, and a clean, well-thought-out exploration agent can land near the top of the board today. If you like RL, agents, and open problems, start here. The agent SDK and docs live at docs.arcprize.org.

Part III: Practical Guide

Your First ARC Solver: A Step-by-Step Tutorial

This section provides a hands-on tutorial for building a simple, yet complete, ARC solver in Python. We'll use a minimal DSL and simple brute-force search to demonstrate the core logic of the inductive programming paradigm.



Python



DSL



Search



Visualization





Overview



Setup



Visualize



Solver



Run

Building Your First ARC Solver

What We'll Build

A minimal Domain-Specific Language (DSL)

A brute-force search algorithm

Grid visualization tools

A complete end-to-end solver

Learning Objectives

Understand the program synthesis approach

Learn how DSLs constrain search space

Implement search and verification logic

Create submission-ready output

Note: this tutorial is just the core concepts. Real competitive solvers use much larger DSLs, smarter search, and neural guidance.

Going Further: Two Things the Winners Add

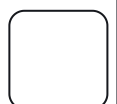
The solver above answers each task once — list a few operations, search for a combination that fits the examples, done. It's the right way to learn the loop, but not how the recent top entries get their scores. The winners keep a base predictor and wrap a generate-check-reconcile loop around it.

augmentation

voting

pass@2

refinement





Overview



Augment & Vote



Validate

Two upgrades to the basic solver

Here are the two simplest pieces of that loop. Both run on a CPU with no trained model, and both map straight onto ideas from the Core Methodologies section — open a tab to see the code (about 20 lines of NumPy each).



Augment & Vote

Show the task to your solver from all eight rotations and flips, undo each, and let the pixels vote. The pattern is called AIRV, and it shows up in nearly every 2024–25 winner.



Validate honestly

A small pass@2 scorer you run before tuning anything, so your local number means something — and so you're watching cost per task from the start.

Why bother: the toy solver answers once and stops. Wrapping even a weak predictor in augmentation + voting + an honest scorer is the cheapest way to make it more reliable — no training required.

Building a Robust Validation Pipeline

Before you write any solver code, build a local validation pipeline. It's the step most people skip, and it's the one that keeps you from fooling yourself about how well you're really doing on the hidden test set.



The Golden Rule of Validation

Your solver should be developed *only* on the public training set. The public evaluation set must be treated as a **one-shot, holdout set** for final validation.

Mimic Kaggle: Your pipeline should strictly separate the public training and public evaluation datasets. Your algorithm should never see the evaluation tasks during its development phase.

Avoid Data Leakage: Repeatedly modifying your algorithm based on its score on the evaluation set, or manually inspecting those tasks to guide development, constitutes data leakage. This will lead to an inflated, unreliable local score that will not translate to the private leaderboard.

One-Shot Execution: To get your best estimate of true performance, run your final, trained solver on the *entire* public evaluation set in a single execution. If your solution is computationally expensive, you can build confidence by testing on a random sample of tasks, holding out the rest for a final validation run before a full execution.

Log your cost, too: on ARC-AGI-2, dollars and seconds per task are part of the score — and the contest box is offline with a hard time limit. Track compute from day one, so a solver that "works" but would blow the budget doesn't surprise you at submission time.

Deconstructing the Champions: Analysis of Winning Solutions

The fastest way to get good at this is to read the winners' code, and ARC makes that easy — everyone has to open-source. Below are two cohorts worth studying: the **2024** winners (on ARC-AGI-1), who set the template everyone still builds on, and the **2025** winners (on the much harder ARC-AGI-2), who show where the bar is now.

The 2024 class (ARC-AGI-1)

These solutions defined the modern playbook: a fine-tuned LLM, test-time training, clever ways of generating and ranking candidates.



Selected ARC Prize 2024 Winners and Approaches

Rank (Category)	Team / Lead Author	Score	Core Approach	Key Innovation(s)
1st (Kaggle)	the ARChitects	53.5%	LLM-based TTT (Transduction)	Custom DFS sampling, "Product of Experts" scoring.
2nd (Kaggle)	G. Barbadillo	40.0%	Ensemble (Induction + Transduction)	Hybrid solver combining DSL search and LLM prediction.
2nd (Paper)	Akyürek et al.	61.9% (public)	LLM-based TTT (Transduction)	Foundational method for TTT on ARC using LoRA.
Runner-Up (Paper)	Simon Ouellette	N/A	Neurally-Guided Program Synthesis (Induction)	GridCoder: using a Transformer to guide DSL search.



1st Place: the ARChitects

Core Approach: LLM-based TTT (Transduction)



Advanced TTT with custom sampling and "Product of Experts" scoring



 2nd Paper: Akyürek et al.

Core Approach: The TTT Pioneers (Transduction)



Foundational TTT methodology with LoRA and data augmentation

 Runner-Up: Simon Ouellette (GridCoder)

Core Approach: Neurally-Guided Program Synthesis (Induction)



Specialized Transformer guiding DSL search for efficient induction

The 2025 class (ARC-AGI-2)

Same competition, much harder benchmark, and the scores dropped accordingly. What's interesting is that the winners didn't invent a new paradigm — they took the 2024 playbook and either industrialized it or twisted it. Notice the cost column: on ARC-AGI-2, efficiency is part of the score, so these are pennies-per-task, not dollars.

ARC Prize 2025 Winners (ARC-AGI-2, private set)

Rank	Team	Score	Core Approach	The twist
1st	NVARC	24.0% @ ~\$0.20/task	Synthetic data + TTT + voting	Built on the ARChitects' 2024 base, fed it ~266K self-generated, spec-checked puzzles.
2nd	the ARChitects	16.5%	Masked-diffusion LLM	Dropped autoregression; a diffusion model that denoises the grid over ~100 refinement steps.



Rank	Team	Score	Core Approach	The twist
3rd	MindsAI	12.6%	Test-time fine-tuning	Years of TTT engineering: augmentation ensembles, tokenizer dropout, vote.



1st Place 2025: NVARC

Core Approach: Synthetic data + TTT + ensemble voting



The 2024 playbook, industrialized — and the recipe to copy



2nd Place 2025: the ARChitects

Core Approach: 2D-aware masked-diffusion LLM



Ditched autoregression for a model that refines the whole grid at once



3rd Place 2025: MindsAI

Core Approach: Test-time fine-tuning, pushed hard



Pure test-time fine-tuning, pushed about as far as it goes

The 2025 **paper** prizes went to the three tiny-model projects covered earlier — **TRM**, **SOAR**, and **CompressARC**. Between the Kaggle winners and the paper winners, the year's whole message is right there: stack synthetic data, test-time training, and refinement loops; don't just reach for a bigger model.



Charting Your Course: Strategies for ARC Prize 2026

You've got the philosophy, the methods, and the winners. Here's how to actually start competing. The combination that still wins is unglamorous: solid engineering, a strict validation habit, and one or two genuinely new ideas on top.

First, pick your track

The 2026 competition has three doors, and they suit different people. Be honest about which one you are before you sink months into it:

ARC-AGI-2 (\$1M)

An engineering-and-efficiency race on a well-mapped problem. Pick this if you like building systems and squeezing accuracy out of a fixed budget.

ARC-AGI-3 (milestones)

Wide-open agent research where the best score is ~12%. Pick this if you like RL, world models, and being early. A clever agent can lead today.

Paper track (\$450K)

For a genuinely new idea. This is how TRM, SOAR, and CompressARC got rewarded. Pick this if your contribution is conceptual.

The ARC-AGI-2 recipe that works

If you're going for the static-puzzle track, don't start from scratch — reproduce the NVARC stack first, then improve it. No single approach solves everything, so the sta



the art is a hybrid:

1

Small model + a pile of synthetic data

Take an open-weight model, then pretrain on lots of generated puzzles — ReARC generators, the BARC sets, plus your own spec-checked ones. This is NVARC's edge.

2

Test-time training + voting

Fine-tune a LoRA on each task's own (heavily augmented) demos, infer under many symmetries, and majority-vote the result. The proven workhorse.

3

Ensemble + a refinement loop

Pair a transductive solver with an inductive one (they fail on different tasks), and wrap the whole thing in a verify-and-fix loop. Watch your cost per task.

The Frontier: Where Do New Ideas Come From?

Reproducing the winners gets you onto the board, not to 85%. ARC-AGI-2 was built specifically to break today's methods, so the real progress is in attacking the three weaknesses it targets:



Understanding that pixels can *represent* an action or concept, rather than just being a pattern to transform.

Compositional Reasoning

Discovering and applying multiple, interacting rules simultaneously, especially when those rules interact with each other.

Contextual Rule Application

Recognizing the context that determines which rule to apply, moving beyond superficial global patterns.

Ready to Start Your ARC Journey?

The resources and checklist you need to actually start competing in the ARC Prize 2026.



Essential Resources

ARC Prize Website: arcprize.org

Official Discord: [ARC Prize Discord](#)

GitHub Repository: [fchollet/ARC-AGI](https://github.com/fchollet/ARC-AGI)

Kaggle Competition: [ARC Prize 2026](#)

ARC-AGI-3 Agent Docs: docs.arcprize.org



Competition Checklist

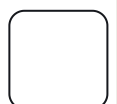
- ☐ Understand the pass@2 metric
- ☐ Build robust validation pipeline
- ☐ Study winning solutions
- ☐ Track cost per task, not just accuracy
- ☐ Open-source ready (CC0 / MIT-0)

Start building →

The Path Forward

ARC-AGI is more than a leaderboard to climb. The bet behind it is that real intelligence isn't the skills you've already banked — it's how efficiently you handle something genuinely new.

What I find motivating is that the gap is still wide open. ARC-AGI-1 fell, but it took crossing it to realize the harder questions — doing this *efficiently*, and doing it in a world you have to explore — were still wide open. ARC-AGI-2's Grand Prize is unclaimed, and ARC-AGI-3 has barely been scratched. Read the winners' code, hang around the Discord, and go after the tasks that break your solver. That's the whole job, and there's plenty of it left.



Ready to start your ARC journey?

